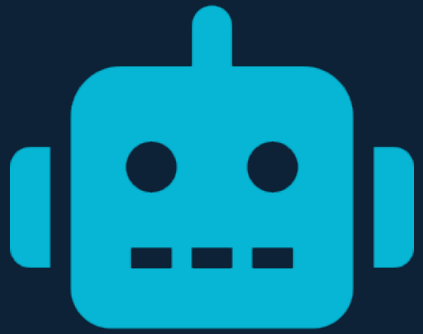

ROBOTICS & AI

LECTURE NOTES

Robot Navigation & Search Algorithms

An in-depth exploration of AMCL · DWA · A Search*

AMCL | DWA | A*

Covering core concepts, challenges, and solutions used in autonomous robot systems.

What We'll Cover

01

Adaptive Monte Carlo Localization

Probabilistic particle-filter localization for mobile robots

02

Dynamic Window Approach

Real-time obstacle avoidance and velocity control

03

A* Search Algorithm

Optimal heuristic-based pathfinding on a cost map

Adaptive Monte Carlo Localization

Particle filters for probabilistic robot localization

Overview & Key Concepts

DESCRIPTION

Adaptive Monte Carlo Localization (AMCL) is a probabilistic algorithm that estimates a robot's position and orientation within a known map. It uses a set of weighted random samples — called particles — each representing a hypothetical pose. Over time, the algorithm refines these particles based on sensor readings (e.g., LIDAR), keeping likely candidates and discarding unlikely ones.

HOW IT WORKS

At each timestep, particles are propagated using the motion model, then reweighted using the sensor model. A resampling step eliminates low-weight particles and duplicates high-weight ones. The 'adaptive' component dynamically adjusts the number of particles based on the estimated uncertainty — fewer particles when confident, more when uncertain.

WHY IT MATTERS

AMCL is a foundational localization technique used in ROS (Robot Operating System) and real-world navigation stacks. It handles the 'kidnapped robot' problem by maintaining a multi-modal belief distribution, allowing recovery from sudden displacements.

The Problem — Where Am I?

CORE CHALLENGE

A mobile robot is placed in a known environment but does not know its exact starting position. GPS is unavailable indoors, and wheel odometry drifts over time. The robot must determine its location using only noisy sensor readings and its map — without direct positional data.

GLOBAL LOCALIZATION

When the robot has no prior pose estimate, it must solve 'global localization': determining its position from scratch in a potentially large or symmetrical environment. Simple sensor matching fails because multiple locations may look identical.

KIDNAPPED ROBOT

If the robot is suddenly moved to a new location (kidnapped), it still believes it is in the old position. A non-adaptive algorithm would never recover — the fixed particle cloud would remain around the wrong location indefinitely, causing catastrophic navigation failures.

The Solution — Particle Filter

PARTICLE REPRESENTATION

AMCL maintains a set of N particles $\{x_1, x_2, ..., x_n\}$, each representing a hypothesis (x, y, θ). Initially spread uniformly across the map, they converge on the true pose as sensor data is collected. Each particle carries a weight proportional to how well it matches current sensor readings.

ADAPTIVE SAMPLING

The KLD-Sampling technique dynamically adjusts N: when uncertainty is high (particle spread is large), more particles are maintained for coverage. As the robot converges to a likely pose, fewer particles are needed. This reduces CPU cost during confident navigation while maintaining robustness during recovery.

RECOVERY BEHAVIOR

When sensor likelihood drops significantly (indicating a possible kidnapping), AMCL injects random particles across the map to explore new hypotheses. This global recovery mechanism allows the robot to re-localize after being displaced, making it robust against real-world perturbations.

Dynamic Window Approach

Real-time local path planning with velocity constraints

Overview & Key Concepts

DESCRIPTION

The Dynamic Window Approach (DWA) is a local obstacle avoidance algorithm for mobile robots. Rather than planning a full path globally, it samples a set of feasible velocity commands in real-time and selects the one that best balances goal-directed progress, obstacle clearance, and velocity efficiency.

VELOCITY SEARCH SPACE

DWA restricts search to a 'dynamic window' in the velocity space (v, ω) — the set of velocities reachable within one time step given the robot's current acceleration limits. Trajectories are simulated forward in time and scored against an objective function to pick the best command.

OBJECTIVE FUNCTION

Each candidate trajectory is evaluated by a weighted combination of: (1) heading alignment toward the goal, (2) clearance from the nearest obstacle, and (3) forward velocity (to favor fast motion). The velocity pair (v, ω) maximizing this score is applied to the robot.

The Problem — Navigating Safely in Real Time

OBSTACLE AVOIDANCE AT SPEED

Global planners produce paths that assume a static environment. When dynamic obstacles (people, moving objects) appear, the global path becomes unsafe. A robot needs to react locally, in milliseconds, while still moving toward its goal — without recalculating a full global plan.

ACTUATOR CONSTRAINTS

Robots have physical limits on acceleration. Commanding an abrupt velocity change can slip wheels, damage actuators, or cause instability. A naive controller might issue commands that are dynamically infeasible, leading to poor tracking or mechanical failure.

DEADLOCK & OSCILLATION

In cluttered environments, simple reactive controllers get stuck oscillating between obstacles or freeze in deadlock. The local planner must be smart enough to navigate tight corridors, doorways, and crowds while keeping smooth, predictable motion.

DYNAMIC WINDOW APPROACH

The Solution — Velocity Space Sampling

DYNAMIC WINDOW SAMPLING

https://www.youtube.com/watch?v=tNtUgMBCh2g

DWA restricts the search space to velocities (v, ω) that are: (a) physically feasible given max acceleration limits, and (b) safe — the robot can come to a complete stop before hitting any obstacle. This guarantees only collision-free, mechanically valid commands are issued.

TRAJECTORY SIMULATION

For each sampled (v, ω) pair, the robot's circular arc trajectory is simulated forward for a fixed time horizon (e.g., 3 seconds). Trajectories that intersect obstacles are discarded. The remaining ones are scored by the objective function, and the best velocity pair is executed immediately.

FAST REPLANNING

DWA runs at 10–20 Hz, enabling rapid response to environmental changes. It integrates seamlessly with a global path planner: the global planner provides the target direction, while DWA ensures safe, smooth execution locally. Together, they form a complete navigation stack.

A* Search Algorithm

Optimal heuristic-guided pathfinding on a cost map

Overview & Key Concepts

DESCRIPTION

A* (A-star) is a best-first graph search algorithm that finds the lowest-cost path from a start node to a goal node. It combines Dijkstra's algorithm (which guarantees shortest path) with a heuristic function h(n) that estimates the remaining cost to the goal, allowing it to search efficiently in large spaces.

EVALUATION FUNCTION

A* evaluates nodes using f(n) = g(n) + h(n), where g(n) is the actual cost from start to node n, and h(n) is the heuristic estimate from n to the goal. By always expanding the node with the lowest f(n), A* finds the optimal path when the heuristic is admissible (never overestimates).

ADMISSIBLE HEURISTICS

Common heuristics include Euclidean distance (straight-line cost) for grid maps and Manhattan distance for 4-connected grids. A well-designed heuristic dramatically reduces the number of nodes explored compared to uninformed Dijkstra search, making A* the standard for robot global planning.

The Problem — Finding the Optimal Path

COMBINATORIAL EXPLOSION

A robot's environment is discretized into thousands or millions of grid cells. Exhaustively searching all possible paths (brute-force BFS or DFS) is computationally infeasible for large maps. A global planner must find the optimal path in milliseconds to support real-time navigation.

OBSTACLES & COST MAPS

Environments contain static obstacles (walls, furniture) and inflated cost zones around them (to protect the robot body). A simple shortest-path algorithm ignores cost gradients. The planner must account for both impassable cells and high-cost regions that should be avoided unless necessary.

SUBOPTIMAL PATHS

Greedy best-first search (using only h(n)) is fast but often produces unnecessarily long or obstacle-hugging paths. Pure Dijkstra is optimal but explores too many nodes. The challenge is finding a path that is simultaneously optimal, fast to compute, and practical for the robot to execute.

The Solution — Heuristic-Guided Optimal Search

OPEN & CLOSED LISTS

A* maintains an open list (priority queue of nodes to explore, sorted by f(n)) and a closed list (already-evaluated nodes). At each step, the node with the lowest f(n) is expanded, its neighbors are evaluated, and their costs updated if a cheaper path is found. This ensures optimality without redundant exploration.

HEURISTIC EFFICIENCY

By incorporating h(n), A* focuses exploration toward the goal rather than expanding uniformly in all directions. With an Euclidean distance heuristic on a 2D grid, A* typically explores far fewer nodes than Dijkstra — often achieving 10x or greater speedup — while still guaranteeing the optimal path.

INTEGRATION WITH NAVIGATION

In ROS navigation stacks, A* runs on the global costmap (a 2D grid with obstacle inflation) to produce a global path from the robot's current position to the goal. This path is then handed to a local planner like DWA for real-time obstacle avoidance during execution.

Summary

AMCL

Uses particle filters to probabilistically estimate a robot's pose. The adaptive mechanism balances computational cost with localization confidence, and supports recovery from sudden displacements.

Dynamic Window Approach

Samples feasible velocity commands in real time and selects the one maximizing a trade-off between goal progress, obstacle clearance, and speed — ensuring dynamically feasible and safe motion.

A* Search

Uses f(n) = g(n) + h(n) to guide search toward the goal efficiently. With an admissible heuristic, it finds the globally optimal path while exploring far fewer nodes than Dijkstra.

Questions?

Thank you for your attention

AMCL

DWA

A*